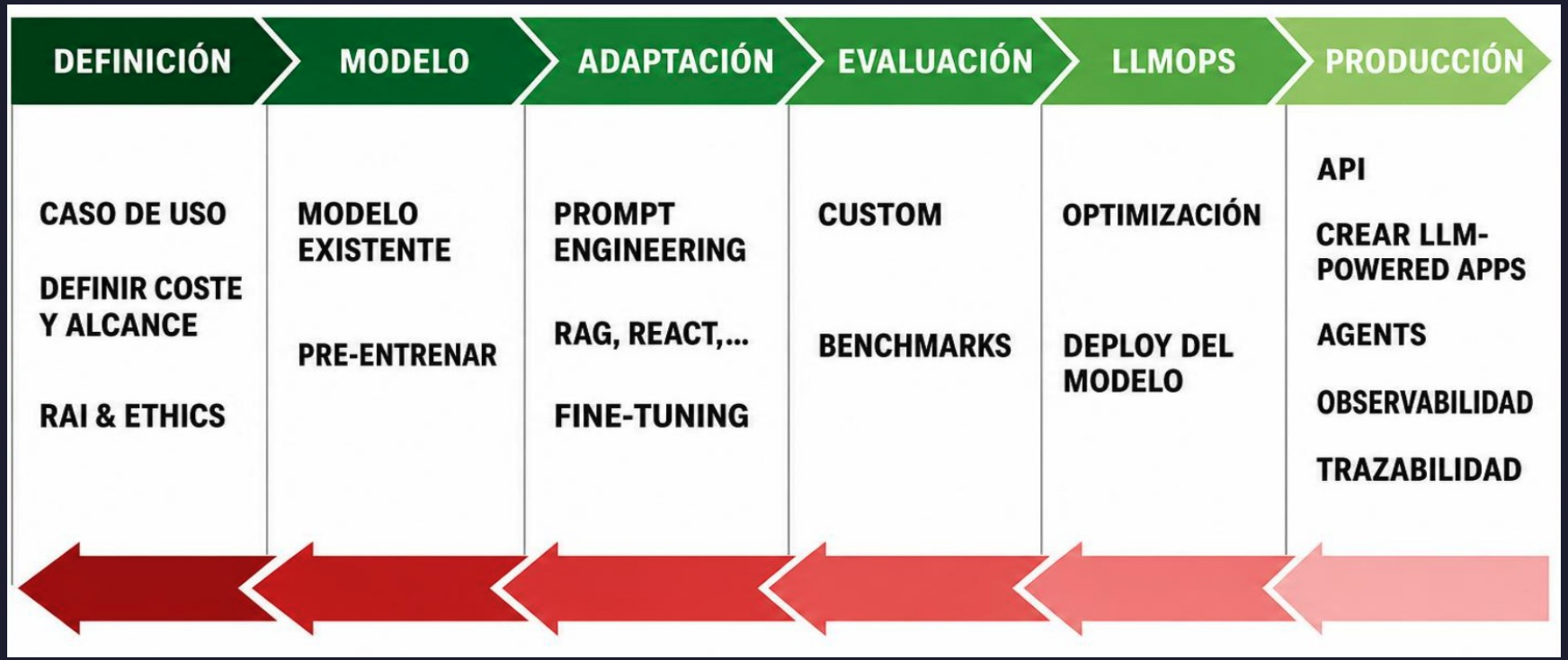
keep coding

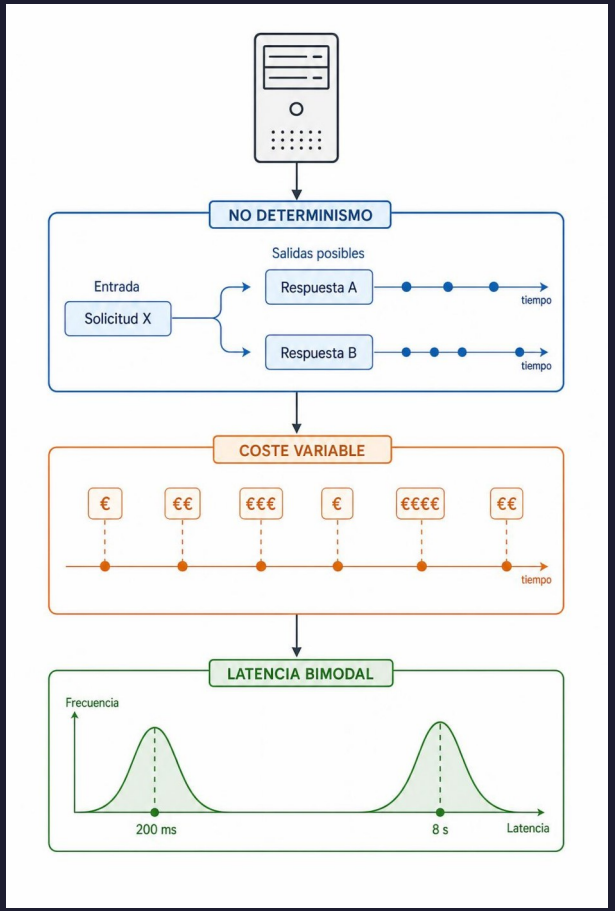
# AI ENGINEERING

## Observabilidad


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

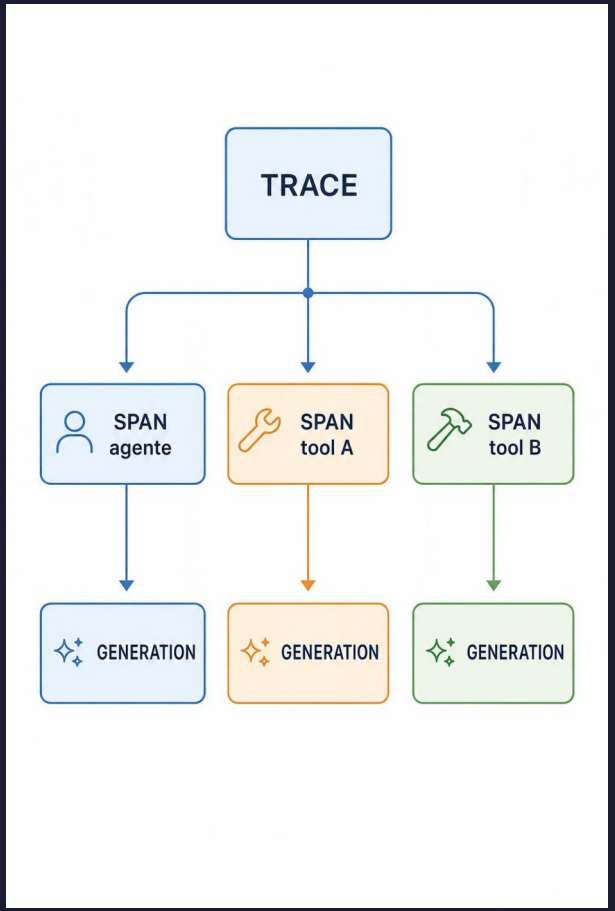
# El problema

- Una LLM app no es un microservicio cualquiera
- No determinismo: misma entrada, distinta salida
- Coste variable: cada llamada cuesta tokens reales
- Latencia bimodal: 200 ms con caché u 8 s en frío
- Una conversación = 5-50 spans encadenados



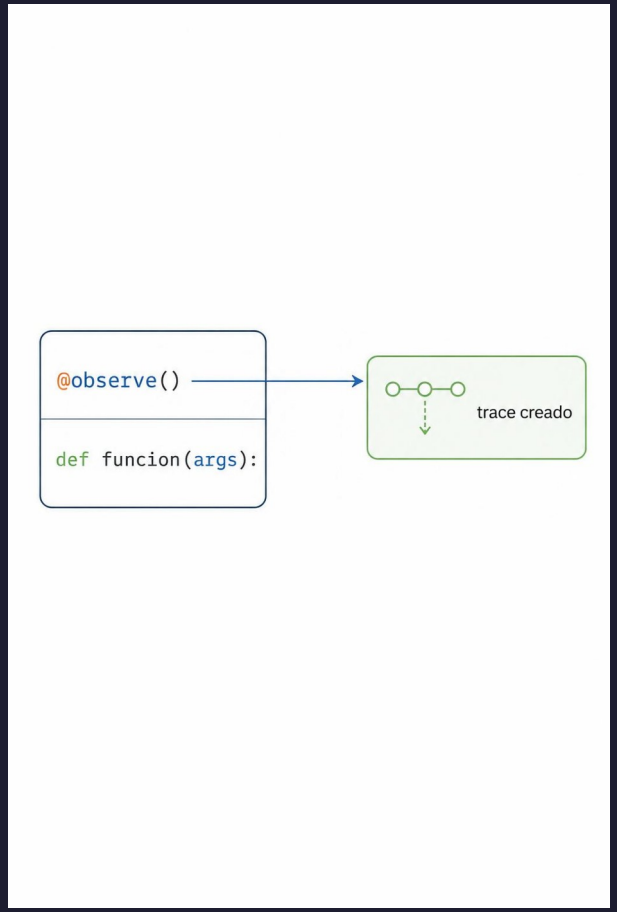
# Trace = árbol de spans

- Una conversación = un trace
- Cada llamada (LLM, tool, retriever) = un span
- Generations: spans de llamadas al modelo
- Padre/hijo: el agente llama a una tool que llama al LLM
- Tokens y coste capturados en cada nivel



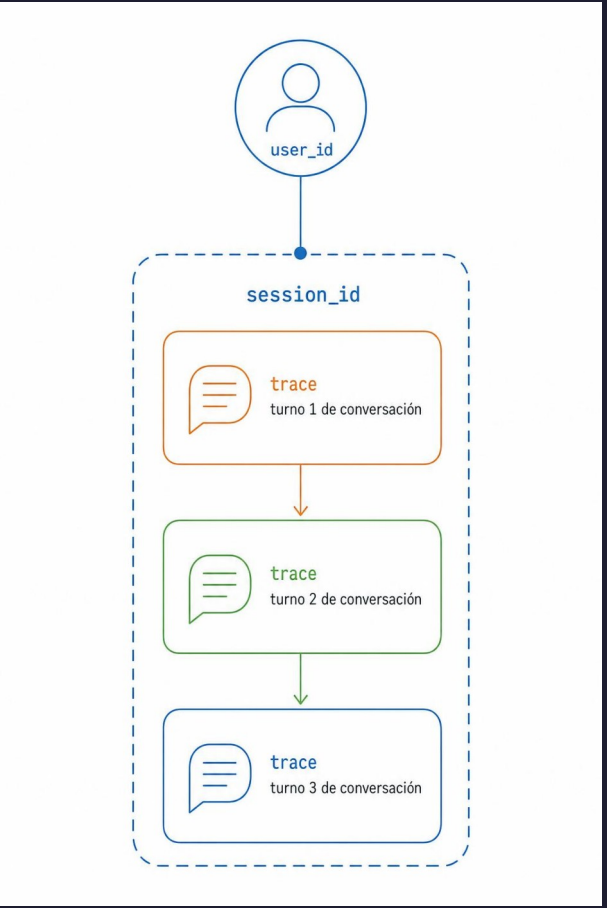
# @observe() en código

- Decorador sobre una función: captura inputs y outputs
- Anidación automática: árbol de spans
- as_type='generation' para llamadas al modelo
- Con LangChain/LangGraph: CallbackHandler
- Compatible con código síncrono y asíncrono



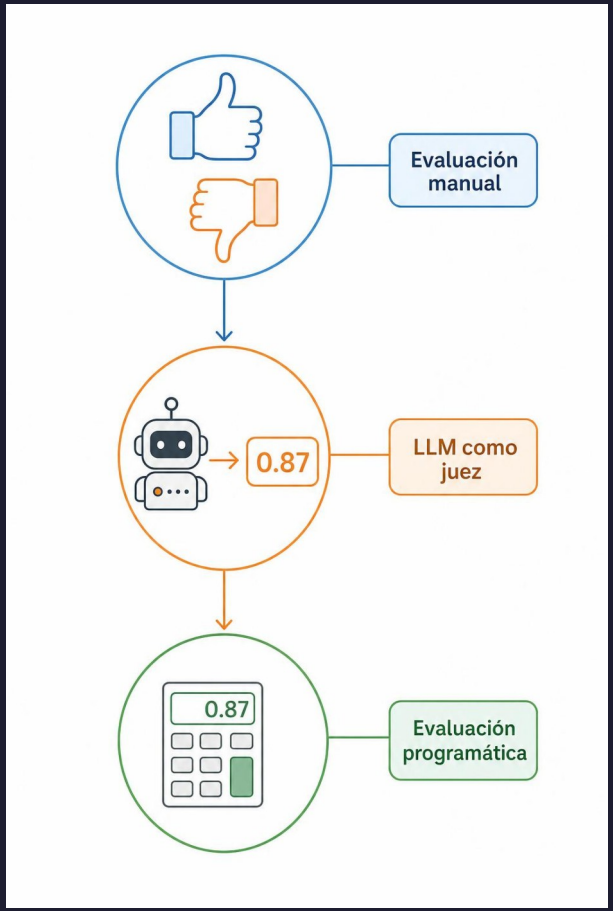
# Sessions, users y metadata

- session_id agrupa la conversación
- user_id agrupa por usuario final
- tags y metadata para filtrar en el dashboard
- Obligatorios en producción
- Sin esto no puedes investigar nada



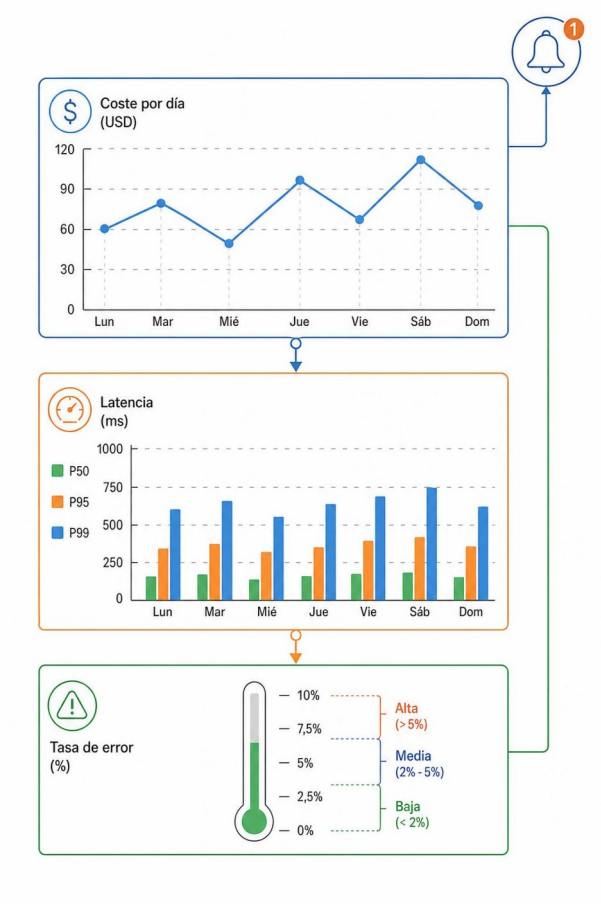
# Scores: la moneda de calidad

- Manuales: pulgar arriba/abajo en la UI
- LLM-as-judge: otro LLM evalúa (clase-3 M3)
- Programáticos: latencia, longitud, regex
- Asociados a un trace, span o generation
- Base de la evaluación automática



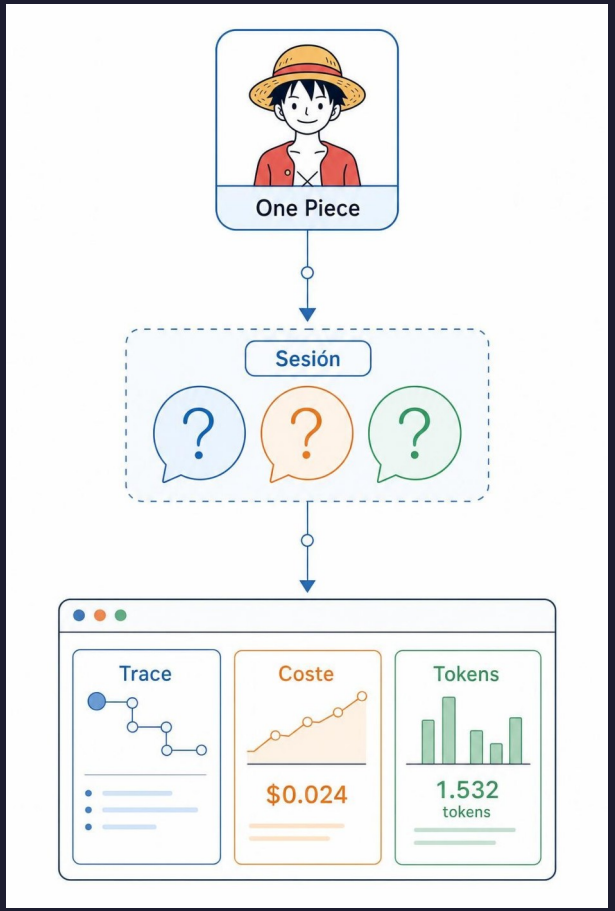
# Dashboards y alertas

- Coste por día, modelo y usuario
- Latencia P50, P95 y P99
- Tasa de error por tool
- Outliers: traces lentos o caros
- Alertas a Slack o Telegram por webhook



# Lab: agente + Langfuse

- Instrumentamos el agente de recompensas de M1
- 3 preguntas en una misma sesión
- Trace, coste y tokens en el dashboard
- Lab: 01-instrumentar-agente-langfuse.ipynb
- Siguiente: M8 · Safety




keep coding